React – Basic and Detailed Explanation

1. What is React?

React is a JavaScript library used for building user interfaces, especially modern and interactive web applications.

React was originally developed by **Facebook**, now known as Meta, and is maintained as an open-source project.

React is mainly used in **frontend development**.

The basic relationship is:

HTML → Structure
CSS → Styling
JavaScript → Logic
React → Building interactive and reusable user interfaces

---

2. Why is React used?

Traditional websites can become difficult to manage when they contain many interactive sections.

React helps developers build applications using **small, reusable components**.

For example, an e-commerce website can be divided into:

* Header
* Navigation bar
* Product card
* Product list
* Shopping cart
* Login form
* Footer

Each part can be developed as a reusable component.

This makes large applications easier to develop and maintain.

---

3. Is React a programming language?

No. React is not a programming language.

React is a **JavaScript library**.

It uses JavaScript to create user interfaces.

Therefore, learning JavaScript before React is very important.

A good learning order is:

HTML → CSS → JavaScript → React

---

4. What is a User Interface?

A **User Interface**, or UI, is the part of an application that users see and interact with.

Examples include:

* Buttons
* Menus
* Forms
* Cards
* Images
* Search boxes
* Navigation bars
* Dashboards
* Login screens

React helps developers create these interface elements in an organized and reusable way.

---

5. What is a Component?

A **component** is one of the most important concepts in React.

A component is a reusable part of a user interface.

For example, an e-commerce application might have components such as:

* Header component
* Product component
* Product card component
* Search component
* Cart component
* Login component
* Footer component

Instead of creating the same design repeatedly, developers can create a component once and reuse it.

---

6. Why are components important?

Components make applications:

* Reusable

* Organized
* Easier to maintain
* Easier to test
* Easier to update

For example, if an application contains 50 product cards, developers can use the same product-card component for all products while providing different product information.

---

7. Component-based architecture

React follows a **component-based approach**.

A complete webpage can be divided into smaller components.

For example:

Website

→ Header
→ Navigation
→ Main Content
→ Product Section
→ Product Card
→ Sidebar
→ Footer

This approach makes complex applications easier to manage.

---

8. JSX

JSX stands for JavaScript XML.

JSX is a syntax commonly used with React that allows developers to describe the structure of user interfaces using a syntax that looks similar to HTML.

Although JSX looks like HTML, it is processed as part of JavaScript.

JSX makes React interfaces easier to understand and organize.

---

9. React and HTML

React does not replace HTML completely.

Instead, React uses JSX to describe the user interface, which ultimately results in browser elements being displayed.

Therefore, having a strong understanding of HTML is important before learning

React.

---

10. React and CSS

React and CSS have different responsibilities.

React → Structure, components, and UI behavior

CSS → Appearance and styling

React applications can be styled using:

* Traditional CSS
* CSS modules
* Tailwind CSS
* Bootstrap
* Other styling solutions

Since you are learning Tailwind CSS and Bootstrap, both can be used with React.

---

11. Props

Props, short for properties, are used to pass information from one component to another.

Props allow components to be reusable with different information.

For example, the same product-card component can display:

* Product A
* Product B
* Product C

Each product can have different:

* Name
* Image
* Price
* Description

Props make this possible.

---

12. State

State is another important React concept.

State represents information that can change while the application is running.

Examples include:

* Shopping cart quantity
* Login status
* Search text
* Selected product
* Menu open or closed
* Counter value
* Form information

When state changes, React can update the relevant part of the user interface.

---

13. Props vs State

This is an important distinction.

Props

Used to **pass information into a component**.

State

Used to **store information that can change within an application or component**.

A simple way to remember:

Props → Data received

State → Data managed and changed

---

14. Events in React

React applications need to respond to user actions.

Examples include:

* Clicking a button
* Typing into an input
* Selecting an option
* Submitting a form
* Moving the mouse
* Opening a menu

React uses event handling to respond to these actions.

---

15. Hooks

Hooks are special React features that allow components to use important React functionality.

Hooks are one of the most important modern React concepts.

Common hooks include:

* useState
* useEffect
* useContext
* useRef
* useMemo
* useCallback

For beginners, **useState and useEffect** are particularly important.

---

16. useState

useState is commonly used to manage changing information inside a React component.

For example, it can manage:

* Counter values
* Form fields
* Menu state
* Selected items
* Shopping cart information

When the state changes, React can update the relevant user interface.

---

17. useEffect

useEffect is commonly used when a component needs to perform certain actions related to external systems or changes.

It can be used for tasks such as:

* Fetching data
* Working with APIs
* Setting up subscriptions
* Performing actions after rendering
* Synchronizing information

Understanding useEffect is important for real-world React applications.

---

18. Virtual DOM

One of the commonly discussed React concepts is the **Virtual DOM**.

The DOM represents the structure of a webpage.

React maintains a lightweight representation of the UI and uses it to determine what needs to be updated.

Instead of unnecessarily updating everything on a webpage, React can efficiently update the relevant parts of the interface.

This contributes to efficient UI rendering.

---

19. Rendering

Rendering means displaying the user interface based on the current data and state.

When relevant information changes, React can re-render the necessary components so that the user sees the updated information.

For example, when the quantity of a product changes, the displayed quantity and total price can update.

---

20. Conditional Rendering

React can display different UI elements depending on a particular condition.

For example:

If a user is logged in:

Show Dashboard

If the user is not logged in:

Show Login

This is called **conditional rendering**.

It is commonly used in real-world applications.

---

21. Lists in React

Applications frequently need to display collections of data.

Examples include:

* Products

* Students
* Employees
* Messages
* Notifications
* Orders

React can use JavaScript data structures such as arrays to generate lists of interface elements.

---

22. Keys

When React displays lists, each item should have a suitable **key**.

Keys help React identify individual elements and understand which items have changed, been added, or removed.

Using appropriate keys is an important React development practice.

---

23. Forms in React

Forms are very common in web applications.

React can be used to manage:

* Login forms
* Registration forms
* Search forms
* Contact forms
* Payment forms
* Profile forms

React can track the information entered by users and respond to form events.

---

24. API Integration

Modern applications often need information from a backend server.

React can communicate with APIs to retrieve or send information.

For example, a React application can obtain:

* Product data
* User data
* Weather data
* Flight information
* News
* Orders

A typical flow is:

React → API → Backend → Database

The backend can communicate with a database such as MySQL or PostgreSQL.

---

25. React and SQL

React normally does **not directly connect to a SQL database**.

A safer and more common architecture is:

React → Backend/API → SQL Database

For example:

1. React requests product information.
2. The backend receives the request.
3. The backend communicates with the SQL database.
4. SQL retrieves the required information.
5. The backend sends the result back.
6. React displays the information.

This is important to understand if your goal is full-stack development.

---

26. React Router

React applications often need multiple pages or views.

React Router is a commonly used library for handling navigation and routing in React applications.

It can be used for routes such as:

* Home
* About
* Products
* Product details
* Login
* Profile
* Dashboard
* Contact

Routing allows users to navigate through different parts of an application.

---

27. Single Page Application

React is commonly used to create **Single Page Applications**, often called

SPAs.

In an SPA, the application can dynamically update the displayed content without requiring a complete traditional page reload for every navigation action.

Examples of applications that can use this approach include:

* Dashboards
* Social platforms
* E-commerce applications
* Project management tools

---

28. React and JavaScript

React depends heavily on JavaScript.

Before learning React, you should understand:

* Variables
* Data types
* Functions
* Arrow functions
* Arrays
* Objects
* Array methods
* Conditions
* Loops
* Destructuring
* Spread operator
* Modules
* Promises
* Async and await
* APIs

Without a good JavaScript foundation, React can be difficult to understand.

---

29. React and Bootstrap

React can be used together with Bootstrap.

Bootstrap provides:

* Ready-made UI components
* Responsive layouts
* Buttons
* Cards
* Forms
* Navigation components

React provides:

* Components
* State management
* Dynamic UI
* Application structure

Together, they can be used to create modern web applications.

---

30. React and Tailwind CSS

React can also be used with Tailwind CSS.

Tailwind provides utility-based styling, while React provides component-based UI development.

This combination is popular for creating customized modern interfaces.

A simple way to remember:

React → Build the UI

Tailwind → Style the UI

---

31. React Advantages

Reusable components

Components can be reused throughout an application.

Efficient UI updates

React efficiently updates relevant parts of the interface.

Large ecosystem

There are many libraries and tools available.

Component-based development

Large applications can be divided into manageable pieces.

Strong community

React has a large developer community and extensive learning resources.

Good for modern applications

React is suitable for interactive and dynamic web applications.

---

32. React Disadvantages

React also has some challenges.

Requires JavaScript knowledge

Beginners who do not understand JavaScript may find React difficult.

Ecosystem can be confusing

There are many additional libraries and tools, which can make technology choices difficult.

Frequent changes

The React ecosystem evolves over time, so developers need to keep their knowledge updated.

Additional tools may be required

For complete applications, developers may need routing, state management, API handling, and other tools.

---

33. React vs JavaScript

JavaScript is the **programming language**.

React is a **JavaScript library**.

JavaScript can be used without React.

React depends on JavaScript.

For example:

JavaScript → Provides programming logic

React → Provides a structured way to build user interfaces using JavaScript

---

34. React vs HTML

HTML provides the basic structure of webpages.

React provides a component-based approach for building dynamic user interfaces.

You should learn HTML before React because React interfaces are built around concepts that ultimately represent webpage elements.

---

35. React vs Bootstrap

They are not direct replacements for each other.

React → UI library for building interactive interfaces

Bootstrap → CSS framework for styling and responsive components

They can be used together.

---

36. React vs Tailwind CSS

They also have different purposes.

React → Component-based UI development

Tailwind CSS → Styling and responsive design

They are commonly used together.

---

37. React Project Structure

A React application can contain different types of files and folders for:

* Components
* Pages
* Styles
* Images
* Services
* API communication
* Configuration
* Assets

Good project organization becomes increasingly important as an application grows.

---

38. React Development Tools

React development commonly involves tools such as:

* Node.js
* npm
* Package managers
* Code editors
* Browser developer tools
* Build tools

These tools help developers create, run, test, and maintain React applications.

---

39. React Ecosystem

React itself focuses mainly on user interfaces.

For a complete application, developers may use additional technologies for:

* Routing
* State management
* API communication
* Authentication
* Form handling
* Styling
* Testing
* Build and deployment

This collection of technologies is commonly referred to as the **React ecosystem**.

---

40. React Learning Roadmap

Since you are learning frontend development, I recommend learning React in this order.

Level 1 — Prerequisites

First learn:

* HTML
* CSS
* JavaScript

Especially focus on modern JavaScript.

Level 2 — React Basics

Learn:

* What React is
* Components
* JSX
* Props
* State
* Events
* Rendering
* Conditional rendering
* Lists
* Keys

Level 3 — React Hooks

Learn:

* useState
* useEffect
* useContext
* useRef
* Other commonly used hooks

Level 4 — Application Development

Learn:

* Forms
* API integration
* Routing
* Authentication
* Loading states
* Error handling

Level 5 — Advanced React

Learn:

* Component architecture
* State management
* Performance optimization
* Reusable components
* Custom hooks
* Testing
* Application structure

---

41. React Project Ideas

After learning the basics, you can practice by creating:

Beginner

* Calculator
* To-do list
* Digital clock
* Simple portfolio
* Weather interface

Intermediate

* E-commerce website
* Student management system
* Expense tracker
* Movie application
* Blog application

Advanced

* Full-stack e-commerce application
* Learning management system
* Job portal
* Project management dashboard
* Flight booking application

---

42. React in your complete learning roadmap

Based on the frontend technologies you are learning, a good progression is:

HTML → CSS → Bootstrap → Tailwind CSS → JavaScript → React → SQL → Backend → Full Stack

HTML

Provides **structure**.

CSS

Provides **design and layout**.

Bootstrap

Provides **ready-made responsive components**.

Tailwind CSS

Provides **utility-based styling**.

JavaScript

Provides **programming logic and interaction**.

React

Provides **component-based user interface development**.

SQL

Provides **database management**.

Backend

Connects the frontend with databases and server-side functionality.

Full Stack

Combines frontend, backend, and database technologies.

---

43. Simple real-world example

Consider an **online shopping website**.

HTML

Defines the basic webpage structure.

CSS

Controls the visual appearance.

Tailwind CSS or Bootstrap

Helps create responsive and attractive UI components.

JavaScript

Handles application logic.

React

Creates reusable components such as product cards, navigation, cart, and checkout interfaces.

Backend

Processes orders, authentication, payments, and business logic.

SQL

Stores users, products, orders, and other data.

Together, these technologies can form a complete modern web application.

---

44. Simple definition to remember

React is a JavaScript library used to build reusable, interactive, and dynamic user interfaces for modern web applications.

Easy way to remember:

HTML = Structure

CSS = Design

Bootstrap = Ready-made UI

Tailwind CSS = Custom utility-based styling

JavaScript = Logic

React = Reusable interactive UI

SQL = Database

Backend = Connects everything together